

Incorrect BlockPos getSquaredDistance() calculation

MC-248225 Chunk loading

Status	Resolved / Fixed
Affected	1.14.4, 1.18.1, 22w03a
Fixed in	22w05a
Reported	2022-01-22

STEPS TO REPRODUCE

1. Launch Minecraft 1.18.1.
2. Download the attached world ZIP.
3. Extract and open the supplied world from the Singleplayer menu.
4. Follow the instructions included with the downloaded world.
5. Observe the directional bias caused by the incorrect `BlockPos` distance calculation, including behavior that can favor negative coordinates instead of selecting the expected closest position.

OBSERVED BEHAVIOUR

Actual behavior

`BlockPos` inherits `Vec3i# getSquaredDistance(Vec3i)`, whose default overload calls:

```
```java
return this.getSquaredDistance(vec, true);
```
```

With `treatAsBlockPos` set to `true`, the method adds `0.5` to the coordinates of the position on which the method is invoked before calculating the squared distance:

```
```java
double d = treatAsBlockPos ? 0.5D : 0.0D;
double e = this.getX() + d - x;
double f = this.getY() + d - y;
double g = this.getZ() + d - z;
return e * e + f * f + g * g;
```
```

Consequently, when the method is called with two `BlockPos` values, the first position is treated as though it were a block center, while the second position is not. For example:

```
```java
BlockPos.ORIGIN.getSquaredDistance(BlockPos.ORIGIN)
```
```

returns `0.75` instead of `0.0`, because the calculation becomes:

```
```text
(0.5 - 0)^2 + (0.5 - 0)^2 + (0.5 - 0)^2 = 0.75
```
```

For two block positions `a` and `b`, the calculation is effectively:

```
```text
(a.x + 0.5 - b.x)^2
+ (a.y + 0.5 - b.y)^2
+ (a.z + 0.5 - b.z)^2
```
```

rather than the squared distance between their integer coordinates. The offset is also applied only to the receiver, making the result dependent on the order in which the positions are passed.

This causes distance checks, radius checks, and distance-based sorting involving `BlockPos` values to use incorrect values. Positions near a threshold can incorrectly be considered inside or outside a range, and candidates can be sorted in the wrong order. The resulting effects include directional biases and incorrect choices in systems such as chunk update ordering, pathfinding and mob restrictions, point-of-interest searches, raid and portal location searches, structure searches, feature placement, and geode generation.

EXPECTED BEHAVIOUR

When calculating the squared distance between two `BlockPos` values, Minecraft should use their actual integer coordinate differences without adding a 0.5-block offset. For example, `BlockPos.ORIGIN.getSquaredDistance(BlockPos.ORIGIN)` should return `0.0`. Distance comparisons, sorting, range checks, pathfinding, rendering, structure searches, POI selection, and world generation should therefore use accurate, directionally unbiased distances and select only the positions that are genuinely closest or within range.

ENVIRONMENT

Minecraft Java Edition Versions: 1.14.4, 1.18.1, 22w03a (includes snapshots)

ORIGINAL DESCRIPTION

The Bug

Calculation of `BlockPos.getSquaredDistance()` is incorrect; the method is used to get the distance between two `BlockPos`, which return offset. This method was not intended to be used against two `BlockPos`, although it is used everywhere. Recent code like `Sculk Sensors` does it correctly by specifying that it should not be treated as a `BlockPos`. This results in issues across several parts of the game, including geode generation, chunk rendering, points of interest (POI) and pathfinding in general.

Attached below is an image visualizing the current broken calculation and the expected calculation.

Code Analysis

Code analysis using 1.18.1 Yarn Mappings.

If you look at this code:

```
BlockPos.ORIGIN.getSquaredDistance(BlockPos.ORIGIN)
```

When run you would expect the result to be 0, although the value is actually 0.75.

This is due to `getSquaredDistance` automatically setting `treatAsBlockPos` to true, which in this case shouldn't happen since it's already a `BlockPos`. Here's the code for `getSquaredDistance`:

```
net.minecraft.util.math.Vec3i.class
public double getSquaredDistance(Vec3i vec) {
    //By default: treatAsBlockPos is true
    return this.getSquaredDistance(vec, true);
}

public double getSquaredDistance(Position pos, boolean treatAsBlockPos) {
    return this.getSquaredDistance(pos.getX(), pos.getY(), pos.getZ(), treatAsBlockPos);
}

public double getSquaredDistance(Vec3i vec, boolean treatAsBlockPos) {
    return this.getSquaredDistance((double)vec.x, (double)vec.y, (double)vec.z, treatAsBlockPos);
}

public double getSquaredDistance(double x, double y, double z, boolean treatAsBlockPos) {
    //Add 0.5D if it should be converted to a blockpos
    double d = treatAsBlockPos ? 0.5D : 0.0D;
    double e = (double)this.getX() + d - x;
    double f = (double)this.getY() + d - y;
```

```
double g = (double)this.getZ() + d - z;  
return e * e + f * f + g * g;  
}
```

As you can see. Asking for the distance between two BlockPos adds 0.5D, which it should not do since BlockPos is already a BlockPos...

Fix

The fix is simply to override getSquaredDistance in BlockPos so treatAsBlockPos is set to false by default.

Alternatively, you could just change the default to false, since the default

MANUAL RESULT

| | |
|------------------|--------------------|
| Version used | |
| Reproduced? | YES / NO / PARTIAL |
| Crash file | |
| Notes | |
| | |
| Tested by / date | |

Live issue: <https://bugs.mojang.com/browse/MC/issues/MC-248225>